

Zero to Hero Study Handbook: Local Agentic Enterprise Platform (LAEP)

Module 1: Foundations & Architecture

1.1 What this project does

Local Agentic Enterprise Platform (LAEP) is a full-stack business automation system with:

- FastAPI backend (``backend/app``)
- Next.js frontend (``frontend/app``)
- Local LLM integration through Ollama (``backend/app/services/ollama_client.py``)
- Multi-agent task execution (``backend/app/services/supervisor.py``, ``backend/app/agents/business_agents.py``)
- Workflow orchestration with approvals (``backend/app/services/workflow_runtime.py``, ``backend/app/services/approval_service.py``)
- SQL generation and safety controls (``backend/app/services/sql_service.py``, ``backend/app/services/sql_safety.py``)
- Reporting exports (PDF/XLSX/PPTX) (``backend/app/services/report_service.py``)
- Observability and governance (metrics, audit logs, rate limits in ``backend/app/main.py`` and ``backend/app/core/*``)

Primary use cases in code:

- Email summary/draft generation (``/api/v1/email/*``)
- Calendar conflicts, agenda, summaries (``/api/v1/calendar/*``)
- Workflow definition and execution (``/api/v1/workflows/*``)
- Human approval gates (``/api/v1/approvals/*``)
- Safe SQL assistant (``/api/v1/sql/*``, ``/api/v1/database/*``)
- Report creation/export (``/api/v1/reports/*``)
- Document ingestion and knowledge search (``/api/v1/documents/*``, ``/api/v1/knowledge/*``)
- Memory and settings persistence (``/api/v1/memory/*``, ``/api/v1/settings/*``)

1.2 Core paradigms and patterns used here

1. ****Layered architecture****

Definition: Separating transport (API), business logic (services), and persistence (models/DB).

In this repo:

- API layer: ``backend/app/api/routers/*.py``
- Service layer: ``backend/app/services/*.py``
- Domain/data layer: ``backend/app/models/entities.py``, ``backend/app/schemas/*.py``

2. ****Dependency Injection (FastAPI ``Depends``)****

Definition: Endpoint dependencies are declared and injected automatically.

In this repo:

- ``db_session`, `get_current_user`, `require_roles`` in ``backend/app/dependencies.py``

3. **Async I/O architecture**

Definition: Non-blocking I/O for DB and HTTP calls.

In this repo:

- Async SQLAlchemy sessions (``AsyncSession``) in `routes/services`
- Async HTTP with ``httpx.AsyncClient`` in ``OllamaClient``

4. **OOP with Template Method pattern (agents)**

Definition: Base class defines execution template; subclasses provide behavior.

In this repo:

- ``BaseAgent.run()`` in ``backend/app/agents/base.py``
- Specialized agents in ``backend/app/agents/business_agents.py``

5. **Graph-based workflow execution**

Definition: Workflow represented as nodes/edges and executed by traversing graph state.

In this repo:

- ``WorkflowDefinition.graph`` with ``nodes`` and ``edges``
- Runtime engine: ``WorkflowRuntimeService._execute_run()``

6. **Safety-first guardrail pattern**

Definition: Sensitive operations require explicit confirmation or approval.

In this repo:

- Destructive SQL creates approval requests when unconfirmed (``backend/app/api/routes/sql.py``)
- Workflow ``approval`` node pauses at ``WAITING_APPROVAL``

7. **Hybrid retrieval pipeline**

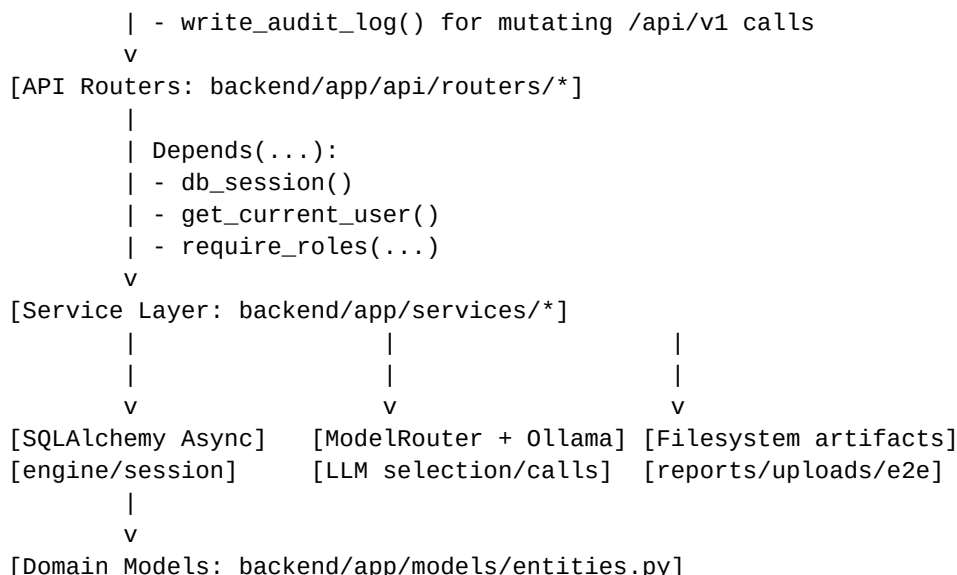
Definition: Combine semantic similarity and keyword scoring.

In this repo:

- ``KnowledgeService.search()`` computes cosine similarity plus keyword bonus

1.3 Architecture and component interaction

```
[Frontend UI: Next.js pages]
|
| apiGet/apiPost (frontend/lib/api.ts)
v
[FastAPI app: backend/app/main.py]
|
| middleware chain:
| - RateLimiter.check()
| - observe_request() + REQUEST_COUNT
```



Module 2: Repository Map

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Configs/Variables
---	---	---	---
`pyproject.toml`	Python dependencies and tool config	N/A	`requires-python`, ruff/mypy/pytest config
`.env.example`	Environment template for backend runtime	N/A	`DATABASE_URL`, `JWT_SECRET`, `OLLAMA_URL`, `BREAK_GLASS_*`, etc.
`docker-compose.yml`	Multi-service deployment topology	N/A	`postgres`, `redis`, `minio`, `api`, `frontend` services
`scripts/bootstrap.sh`	Local Python env bootstrap	shell steps	`uv venv --python 3.12.10`, `uv sync --group dev`
`scripts/checks.sh`	Quality gate script	shell steps	Runs `ruff`, `mypy`, `pytest`
`scripts/run_api.sh`	Backend dev startup	shell steps	`PYTHONPATH=backend`, uvicorn `--reload`
`scripts/run_frontend.sh`	Frontend dev startup	shell steps	`npm run dev -- --port 3000`
`scripts/verify_live_e2e.sh`	End-to-end API/UI verifier	`api_call`, `api_expect_status`, `check_page`	`BASE_URL`, `WEB_URL`, `OUT_DIR`
`backend/app/main.py`	FastAPI app factory, middleware, startup lifecycle	`create_app()`, `lifespan`, `telemetry_middleware`, `audit_middleware`	`settings.frontend_origin`
`backend/app/api/router.py`	Aggregates all API routers	`include_router(...)`	Prefix `/api/v1`
`backend/app/dependencies.py`	Shared DB/auth/role dependencies	`db_session`, `get_current_user`, `require_roles`	JWT bearer extraction via `HTTPBearer`

| `backend/app/core/config.py` | Typed settings loader | `Settings`,
`get\_settings()` | all `.env`-derived keys |

| `backend/app/core/database.py` | Engine/session setup and table init | `engine`,
`SessionLocal`, `init\_database()` | SQLite timeout vs non-SQLite pool settings |

| `backend/app/core/security.py` | Password hashing + JWT encode/decode |
`hash\_password`, `verify\_password`, `create\_access\_token`, `decode\_token` |
`PBKDF2\_ITERATIONS=390000`, `jwt\_secret` |

| `backend/app/models/entities.py` | SQLAlchemy enums/tables | `User`,
`WorkflowRun`, `Report`, `ApprovalRequest`, etc. | `Role`, `WorkflowRunStatus`,
unique/index constraints |

| `backend/app/services/workflow\_runtime.py` | Workflow graph execution engine |
`start\_run`, `\_execute\_run`, `\_execute\_node` | node types and edge condition
evaluation |

| `backend/app/services/sql\_service.py` | NL-to-SQL generation and execution |
`generate`, `execute`, `\_validate\_generated\_sql` | strict prompt retry, sqlglot
validation |

| `backend/app/services/report\_service.py` | Export report files to disk |
`to\_pdf`, `to\_excel`, `to\_pptx` | output dir `artifacts/reports` |

| `backend/app/services/knowledge\_service.py` | Ingest docs and search chunks |
`ingest\_document`, `search` | keyword bonus `0.2` |

| `frontend/app/layout.tsx` | Frontend root composition | `RootLayout` | wraps
providers + `AppShell` |

| `frontend/components/AuthProvider.tsx` | Token state management | `AuthProvider`,
`useAuth` | localStorage key `laep\_token` |

| `frontend/components/AppShell.tsx` | Main navigation and shell UI | `AppShell` |
route map + keyboard shortcuts |

| `frontend/lib/api.ts` | API client wrapper | `API\_BASE`, `apiGet`, `apiPost` |
`NEXT\_PUBLIC\_API\_BASE` |

Module 3: Core Execution Flows

Flow 1: Backend startup, lifecycle, and middleware

****Path:**** `backend/app/main.py`

1. `create\_app()` is called once at import (`app = create\_app()`).
2. `setup\_logging()` configures structured console/file logs.
3. Lifespan startup:
 - `await init\_database()`
 - `await auth\_service.bootstrap\_break\_glass\_user(session)`
4. Middleware chain runs per request:
 - `RateLimiter.check(request)`
 - `observe\_request(path, method)` and `REQUEST\_COUNT.labels(...).inc()`

- audit write for mutating `/api/v1` requests via `write\_audit\_log(...)`

5. `/metrics` returns Prometheus output.

Flow 2: Authentication and authorization

Paths:

`backend/app/api/routers/auth.py`, `backend/app/dependencies.py`, `backend/app/core/security.py`

1. `POST /api/v1/auth/login` with `LoginRequest` (`username`, `password`).
2. `AuthService.authenticate\_local(...)` verifies stored password hash.
3. `create\_access\_token(...)` returns JWT with claims `sub`, `roles`, `exp`, optional `uid`.
4. Protected endpoint resolution through `get\_current\_user(...)`.
5. Role-restricted endpoints use `require\_roles(...)` and can raise HTTP 403.

Input shape:

```
{"username": "local-admin", "password": "change-me-now"}
```

Output shape (`TokenResponse`):

```
{"access_token": "<jwt>", "token_type": "bearer", "roles": ["org_admin"]}
```

Flow 3: Agent execution through supervisor

Paths:

`backend/app/api/routers/agents.py`, `backend/app/services/supervisor.py`, `backend/app/agents/base.py`

1. Client posts `AgentTaskRequest`.
2. Router checks `require\_approval`.
3. If approval required: creates `ApprovalRequest` and returns `status="waiting\_approval"`.
4. Else: `SupervisorService.execute\_agent\_task(...)` dispatches agent and persists `AgentExecution`.

Flow 4: Workflow orchestration and approval pause

Paths:

`backend/app/api/routers/workflows.py`, `backend/app/services/workflow\_runtime.py`

1. `POST /workflows/definitions` saves graph (`nodes`, `edges`).
2. `POST /workflows/runs` creates run and starts async execution task.
3. Runtime computes frontier/indegree, executes nodes, stores timeline in `execution\_timeline.events`.
4. `approval` node creates approval record and sets status `WAITING\_APPROVAL`.

Flow 5: SQL generation, safety analysis, and execution gate

Paths:

``backend/app/api/routers/sql.py`, `backend/app/services/sql_service.py`, `backend/app/services/sql_safety.py``

1. ``/sql/generate`` builds SQL from NL prompt.
2. SQL is validated with ``sqlglot.parse``.
3. ``/sql/execute`` inspects risk:
 - `destructive + `confirm=false` => approval request`
 - otherwise executes query

Generate response shape:

```
{"sql": "SELECT ...", "explanation": "...", "operation": "SELECT", "destructive": false}
```

Flow 6: Report creation and exports

****Paths:****

``backend/app/api/routers/reports.py`, `backend/app/services/report_service.py``

1. ``POST /reports`` inserts report metadata/payload.
2. ``POST /reports/{id}/export/{fmt}`` writes artifact under ``artifacts/reports``.
3. ``report.file_path`` is persisted and returned.

Flow 7: Document ingest and knowledge search

****Paths:****

``backend/app/api/routers/documents.py`, `backend/app/services/knowledge_service.py`,
`backend/app/api/routers/knowledge.py``

1. Upload saves file in ``artifacts/uploads``.
2. Ingest extracts/chunks text and writes embeddings into ``knowledge_chunks``.
3. Search embeds query, ranks by cosine similarity + keyword bonus, returns citations.

Module 4: Setup & Run Guide

4.1 Tech stack inferred from repo

- Backend: Python, FastAPI, SQLAlchemy async, Pydantic
- Frontend: Next.js 15, React 19, TypeScript, Tailwind CSS
- Runtime dependencies: PostgreSQL, Redis, MinIO, Ollama (with SQLite fallback)

4.2 Clean-machine setup

1. Bootstrap backend env:

```
./scripts/bootstrap.sh
```

2. Install frontend dependencies:

```
cd frontend
npm install
```

3. Configure environment:

```
cd ..
cp .env.example .env
```

4.3 Typical startup commands

Backend dev:

```
./scripts/run_api.sh
```

Frontend dev:

```
./scripts/run_frontend.sh
```

Docker stack:

```
docker compose up --build
```

4.4 Required environment variables (`.env` keys)

- `APP\_NAME`, `ENV`, `API\_HOST`, `API\_PORT`, `FRONTEND\_ORIGIN`
- `DATABASE\_URL`, `REDIS\_URL`
- `MINIO\_ENDPOINT`, `MINIO\_ACCESS\_KEY`, `MINIO\_SECRET\_KEY`, `MINIO\_BUCKET`
- `JWT\_SECRET`, `JWT\_ALGORITHM`, `ACCESS\_TOKEN\_MINUTES`, `REFRESH\_TOKEN\_MINUTES`
- `GOOGLE\_CLIENT\_ID`, `GOOGLE\_CLIENT\_SECRET`, `GOOGLE\_REDIRECT\_URI`
- `BREAK\_GLASS\_USERNAME`, `BREAK\_GLASS\_PASSWORD`
- `OLLAMA\_URL`, `OLLAMA\_TIMEOUT\_SECONDS`
- `MODEL\_ROUTER\_DEFAULT\_CHAT`, `MODEL\_ROUTER\_DEFAULT\_EMBED`,
`MODEL\_ROUTER\_DEFAULT\_OCR`
- `ENABLE\_GPU\_AUTO\_DETECT`, `OTEL\_EXPORTER\_OTLP\_ENDPOINT`, `RATE\_LIMIT\_PER\_MINUTE`

4.5 Migration and seeding notes

- Runtime startup creates tables via `init\_database()` (`Base.metadata.create\_all`).
- Runtime startup also seeds/updates break-glass user via `bootstrap\_break\_glass\_user`.
- Versioned migration support exists under `backend/alembic` (`20260628\_0001\_initial.py`).

Module 5: Study Plan & Practice Exercises

5.1 Ordered study plan

1. Manifests/config: `pyproject.toml`, `frontend/package.json`, `.env.example`, `docker-compose.yml`

2. Startup/wiring: ``backend/app/main.py`, `backend/app/api/router.py`, `backend/app/dependencies.py``
3. Core runtime: ``backend/app/core/config.py`, `database.py`, `security.py``
4. Data contracts: ``backend/app/models/entities.py`, `backend/app/schemas/*.py``
5. Orchestration: ``supervisor.py`, `agents/base.py`, `agents/business_agents.py`, `model_router.py``
6. Critical engines: ``workflow_runtime.py`, `sql_service.py`, `sql_safety.py`, `report_service.py`, `knowledge_service.py``
7. API routers by domain: ``auth.py`, `agents.py`, `workflows.py`, `sql.py`, `reports.py`, `database.py``
8. Frontend integration: ``frontend/lib/api.ts`, `AuthProvider.tsx`, `AppShell.tsx`, feature pages`
9. Scripts/tests: ``scripts/checks.sh`, `scripts/verify_live_e2e.sh`, `backend/tests/*``

5.2 Practical exercises with solution outlines

1. **Why can a viewer not create users?**

Read: ``backend/app/api/routers/auth.py`, `backend/app/dependencies.py``

Outline: ``/auth/users`` uses ``require_roles(Role.ORG_ADMIN)``; non-admin gets HTTP 403.

2. **Trace destructive SQL with ``confirm=false``.**

Read: ``backend/app/api/routers/sql.py`, `backend/app/services/sql_safety.py``

Outline: classified destructive, creates approval request, returns ``safe=false`` with ``approval_required`` id.

3. **How does workflow pause on approval?**

Read: ``backend/app/services/workflow_runtime.py``

Outline: approval node triggers ``ApprovalService.create``, runtime sets ``WAITING_APPROVAL``, stops further frontier traversal.

4. **How does model fallback work?**

Read: ``backend/app/services/model_router.py``

Outline: if preferred model unavailable, returns workload default from ``default_map``.

5. **Where are report files written and persisted?**

Read: ``backend/app/services/report_service.py`, `backend/app/api/routers/reports.py``

Outline: artifacts in ``artifacts/reports``, path saved in ``Report.file_path``.

6. **How does document ingest populate knowledge search?**

Read: ``backend/app/services/knowledge_service.py``

Outline: extract text, chunk, embed, store ``KnowledgeChunk``, search by cosine similarity + keyword bonus.

7. **Where is audit logging done and who can read logs?**

Read: ``backend/app/main.py`, `backend/app/api/routers/logs.py``

Outline: mutating ``/api/v1`` requests are logged; ``/logs/audit`` returns data only for ``ORG_ADMIN`/`MANAGER``.

8. **Map one frontend action to backend endpoint.**

Read: ``frontend/app/sql/page.tsx``, ``frontend/lib/api.ts``, ``backend/app/api/routers/sql.py``

Outline: page calls ``/sql/generate`` and ``/sql/execute``; backend validates/safeguards and returns structured JSON.

Understanding Checklist

Use this checklist after studying:

- Can you explain how ``backend/app/main.py`` initializes the app, middleware, and startup lifecycle?
- Can you trace token authentication from ``/auth/login`` through ``get_current_user``?
- Can you explain the role of ``require_roles`` and where RBAC is enforced?
- Can you describe how ``SupervisorService`` selects and runs a specialist agent?
- Can you explain how ``ModelRouter.select()`` chooses models and falls back?
- Can you explain the SQL safety flow from generation to execution approval gate?
- Can you explain workflow graph execution, including frontier logic and approval pauses?
- Can you describe how report files are generated and where paths are stored?
- Can you explain document ingestion and knowledge search scoring logic?
- Can you map one frontend page action to its backend endpoint and response shape?
- Can you list all major ``env`` keys and what subsystem each configures?
- Can you explain when to rely on ``init_database()`` vs Alembic migration commands?